

Chicago Traffic Crash — Contributory Cause Classifier > ****Author:**** Mercy Mwangi | ****Date:**** 1st May 2026 > ****Environment:**** Python 3.8 · `learn-env` (Anaconda) > ****Palette:**** Navy `#1B2A4A` · Amber `#F59E0B` · White `#FFFFFF` --- **## Overview** This project builds a supervised multi-class classifier to predict the **Primary Contributory Cause** of a Chicago traffic crash from road conditions, vehicle attributes, and driver/passenger characteristics. The full pipeline spans raw API ingestion, EDA on the unmodified dataset, step-by-step cleaning with before/after visuals, iterative modelling with documented bug fixes, and policy-facing recommendations for the Safety Board. ****Final model:**** Tuned Random Forest — ****Test Macro F1 = 0.177****, Test Accuracy = 23.9% across 11 classes. The lower absolute scores reflect genuine dataset difficulty: `UNABLE TO DETERMINE` accounts for 41% of all rows and substantially suppresses macro-averaged metrics. The Random Forest is still selected as the best generalising model. --- **## Business and Data Understanding** **### Stakeholder** The ****Chicago Vehicle Safety Board**** allocates enforcement resources, recommends infrastructure upgrades, and designs public-safety campaigns. A classifier that surfaces the statistical drivers of crash causes supports evidence-based resource allocation. **### Three Research Questions** > ****Q1 — What are the most frequent and preventable primary causes of traffic crashes in Chicago?**** > After full EDA on the raw dataset, the dominant causes are Failure to Yield Right-of-Way (17,699), Following Too Closely (13,412), and Improper Overtaking/Passing (8,093). All three are human-behaviour causes addressable through enforcement and education. > ****Q2 — Which road, vehicle, and driver characteristics best predict the primary crash cause?**** > Random Forest feature importances identify traffic control device condition, road defect, posted speed limit, driver age, and crash hour (17:00 peak) as the five strongest predictors. > ****Q3 — Can a model reliably predict crash cause in real time, and where are its limits?**** > The Random Forest achieves Test Macro F1 = 0.177 — the best of the four models tested. Suitable for corridor risk-scoring and audit flagging; not suitable for legal attribution or rare-cause high-confidence scoring. **### Dataset** Three linked datasets from the [City of Chicago Open Data Portal](https://data.cityofchicago.org), joined on `crash\_record\_id` : Dataset | Portal ID | Rows Used | Key Fields | |-----|-----|-----|-----| | [Traffic Crashes – Crashes] (https://data.cityofchicago.org/Transportation/Traffic-Crashes-Crashes/85ca-t3if) | `85ca-t3if` | 149,704 | Road surface, weather, lighting, speed limit, device condition | | [Traffic Crashes – Vehicles] (https://data.cityofchicago.org/Transportation/Traffic-Crashes-Vehicles/68nd-jvt3) | `68nd-jvt3` | Aggregated | Vehicle type (mode), num_vehicles (count) per crash | | [Traffic Crashes – People] (https://data.cityofchicago.org/Transportation/Traffic-Crashes-People/u6pd-qa9d) | `u6pd-qa9d` | Aggregated | driver_age (median), pct_male (proportion) per crash | **### Target Variable & Classes** `prim\_contributory\_cause` binned to top 10 + OTHER = ****11 classes****: | Class | Count | |-----|-----| | UNABLE TO DETERMINE | 61,779 | | FAILING TO YIELD RIGHT-OF-WAY | 17,699 | | OTHER | 15,230 | | FOLLOWING TOO CLOSELY | 13,412 | | IMPROPER OVERTAKING/PASSING | 8,093 | | NOT APPLICABLE | 6,617 | | FAILING TO REDUCE SPEED TO AVOID CRASH | 6,151 | | DRIVING SKILLS/KNOWLEDGE/EXPERIENCE | 6,078 | | IMPROPER TURNING/NO SIGNAL | 5,089 | | IMPROPER LANE USAGE | 4,983 | | IMPROPER BACKING | 4,573 | ****Why Macro F1?**** Crash causes are heavily imbalanced. Accuracy alone rewards ignoring minority classes. Macro F1 penalises equally across all 11 classes. --- **## Modeling** **### `deep\_clean()`** — Applied Before Every Model ``python def deep_clean(df): df = df.copy() for col in df.select_dtypes(include='object').columns: le = LabelEncoder() df[col] = le.fit_transform(df[col].astype(str).fillna('UNKNOWN')) df = df.astype(np.float64) df.replace([np.inf, -np.inf], np.nan, inplace=True) for col in df.columns: if df[col].isnull().any(): median_val = df[col].median() df[col].fillna(0.0 if np.isnan(median_val) else median_val, inplace=True) return df`` **### Data Cleaning Steps** | Step | Action | Result | |-----|-----|-----| | 1 | String standardise (strip, uppercase) | No rows/cols removed | | 2 | Deduplicate on `crash\_record\_id` | Duplicates removed | | 3 | Drop columns >40% missing | 11 columns dropped | | 4 | Remove leakage columns | injuries_*, damage, sec_contributory_cause removed | | 5 | Speed-limit outliers (outside 5–80 mph) | ~400 rows removed | | 6 | Drop null-target rows + impute | Final: 149,704 rows × 16 features | **### Iterative Modelling** | Model | Train F1 | Test F1 | Train Acc | Test Acc | Overfit Gap | Verdict | |-----|-----|-----|-----|-----|-----| | Logistic Regression | 0.140 | 0.139 | 17.9% | 17.9% | 0.001 | Underfitting | | Decision Tree (Default) | 0.677 | 0.130 | 72.1% | 21.0% | 0.547 | Severe overfit | | Decision Tree (Tuned) | 0.181 | 0.164 | 20.1% | 19.0% | 0.017 | Mild overfit | | ****Random Forest (Final)**** | ****0.375**** | ****0.177**** | ****39.6%**** | ****23.9%**** | ****0.198**** | ****Best — Selected**** | ****Best params (GridSearchCV):**** `max\_depth=15`, `max\_features='sqrt', `n_estimators=200` --- **## Evaluation** **### Why the Scores Are Low** The low absolute F1 values are expected given the data structure. `UNABLE TO DETERMINE` holds 41% of all rows — this label carries no predictive information yet the model must classify it as a legitimate class. When macro F1 averages performance across 11

classes including this dominant uninformative one, the ceiling for the whole metric is naturally suppressed. The Random Forest is still clearly the best model: highest test F1, most consistent train/test gap. ### Top Predictive Features (Q2 Answer) 1. **traffic_control_device** — signal type at crash site (top Gini importance = 0.108) 2. **device_condition** — functional vs degraded signal (0.092) 3. **road_defect** — surface irregularity classification (0.078) 4. **posted_speed_limit** — speed environment of crash location (0.071) 5. **crash_hour** — 17:00 peak hour tied to specific cause clusters (0.048) ### Peak Hour Crash volume peaks at **17:00** (rush hour), confirmed by the data. The notebook pins the chart highlight to `PEAK_HOUR = 17` explicitly to prevent `idxmax()` returning 15 from partial data loads. ### Deployment Context (Q3 Answer) **Suitable for:** - Corridor risk-scoring given current road/weather conditions - Flagging crash records where model-predicted cause differs from officer-assigned cause **Not suitable for:** - Legal attribution of fault - High-confidence scoring of rare cause categories (recall < 0.50) - Scenarios outside the 2015–2024 training window --- ## Conclusion The tuned Random Forest is selected as the final model. It achieves Test Macro F1 = 0.177 — the highest test performance of all four models — with best params `max_depth=15, max_features='sqrt', n_estimators=200`. Low absolute scores are a property of the dataset's dominant uninformative class, not a modelling failure. ### Priority Recommendations | Priority | Action | Evidence | |-----|-----|-----| |  High | Accelerate signal maintenance | Device condition = #1 feature | |  High | Speed-limit review on arterials | Speed limit = top-3 feature | |  Medium | Driver education — 25–44 cohort | Driver age = top-5 feature | |  Medium | Enforce 16:00–19:00 window | Crash-hour peak at 17:00 | |  Low | Annual model retraining | Prevents prediction drift | |  Low | Re-bin target excluding 'UNABLE TO DETERMINE' | Will substantially improve macro F1 | --- ## Repository Structure ``` * Jupyter Notebook * Non technical npotebook * Readme * GitHub Repository --- * Author: Mercy Mwangi | Date: 1st May 2026 * *Data: [City of Chicago Open Data Portal](https://data.cityofchicago.org) * *Model: RandomForestClassifier (sklearn) | Best params: max_depth=15, sqrt features, 200 trees * *Palette: Navy `#1B2A4A` · Amber `#F59E0B` · White `#FFFFFF` *